

Chord Flow: Optimal Voice Leading for Chord Progressions via Shortest Paths in Layered Voicing Graphs

Will Dickerson

<https://github.com/willdickerson/chord-flow>

Abstract. Voice leading—the art of connecting successive chords so that individual voices move as little as possible—is a foundational skill in Western harmony and jazz practice, yet computing good voice leadings by hand is tedious for long progressions. We present *Chord Flow*, an open-source, browser-based system that computes globally optimal voice leadings for arbitrary chord progressions. Chord Flow reduces the problem to a single-source shortest-path computation on a layered directed acyclic graph whose nodes are concrete, register-specific voicings of each chord and whose edge weights are a (optionally per-voice weighted) taxicab displacement metric. Two practical refinements distinguish the formulation: a *phantom start layer* that approximates cyclic optimality for looped playback, and an *anti-stasis filter* that suppresses degenerate zero-motion transitions. The system supports both triadic voicings (close and open position) and four-note seventh chords (close inversions and drop-2 voicings), renders results as interactive notation and audio in the browser, and ships with a built-in library of 1,695 jazz standard chord charts. On a ii–V–I progression the optimizer recovers the canonical voice leading taught in harmony pedagogy, and it does so in milliseconds even for 32-bar forms.

Keywords: voice leading, computational musicology, shortest paths, graph algorithms, music software, web audio

1 Introduction

Given a chord progression such as $Dm^7 \rightarrow G^7 \rightarrow Cmaj^7$, a performer must decide not merely *which* pitch classes to play but *where*: each chord admits many *voicings*—assignments of its tones to concrete registers—and the musical quality of the realization depends critically on how the voicings connect. Classical part writing and jazz comping alike prize *efficient voice leading*: successive chords should be voiced so that each voice moves by the smallest possible interval, producing smooth, singable lines and minimizing perceptual disruption [1]. Tymoczko has shown that this preference has deep geometric structure: chords inhabit quotient spaces (orbifolds) in which efficient voice leadings correspond to short line segments [2, 4, 3].

While the two-chord problem—find the minimal voice leading between chords A and B —is well studied, practical musicianship poses the *sequential* version: choose one voicing per chord for an entire progression so that the *total* motion is minimized, subject to register constraints and stylistic voicing vocabularies. Greedy chord-to-chord choices are myopic; a locally smooth move can strand the hands at the top of the instrument three bars later. The sequential prob-

lem is naturally a shortest path problem, and this paper describes a complete, deployed system built on that observation.

*Chord Flow*¹ is a client-side web application that (i) parses chord symbols in common lead-sheet notation, (ii) enumerates a stylistically constrained set of concrete voicings for each chord, (iii) builds a layered graph over those voicings with displacement-based edge weights, and (iv) extracts a globally optimal voicing sequence with Dijkstra’s algorithm [5]. The result is played back with sample-accurate Web Audio synthesis, displayed on an interactive piano keyboard and as engraved notation, and exportable as WAV or MIDI.

Our contributions are:

- a clean formulation of sequential voice-leading optimization over *register-specific* voicing sets, including per-voice weighting that lets users optimize only selected voices (e.g., bass line only);
- a *phantom-layer* construction that biases the solution toward cyclic smoothness, so that looped playback of a song form wraps around gracefully;

¹Live at <https://willdickerson.github.io/chord-flow/>; MIT-licensed source at <https://github.com/willdickerson/chord-flow>.

- an *anti-stasis* edge filter that prevents the degenerate optimum in which a shared voicing is simply repeated across chords;
- an open-source, dependency-light implementation that runs entirely in the browser, with an empirical characterization of graph sizes and solutions on standard progressions.

2 Related Work

The geometry of voice leading has been developed extensively by Tymoczko and collaborators [2, 3, 4], who model chords as points in quotient spaces and voice leadings as paths; our taxicab displacement metric is the L^1 instance of the interval-displacement metrics they axiomatize. Huron [1] derives traditional voice-leading rules from perceptual principles, providing the cognitive motivation for minimizing motion. Rule-based and probabilistic systems have long tackled the related problem of *harmonization*—choosing chords and realizations for a given melody—including Ebcioglu’s CHORAL expert system [6], constraint-programming formulations [7], and HMM-based chorale harmonization with dynamic-programming decoding [8]. Chord Flow addresses a complementary, performance-oriented problem: the chord *symbols* are given (as on a jazz lead sheet), and the system optimizes their registral realization. Its decoding step is the classical shortest-path recursion on a layered graph, equivalent to Viterbi decoding with degenerate (deterministic) emissions. On the systems side, Chord Flow builds on the Web Audio ecosystem, using Tone.js [10] for scheduling and synthesis and abcjs [11] for in-browser engraving.

3 Problem Formulation

3.1 Chords and voicings

A *chord symbol* c (e.g., $F\sharp m7b5$) is parsed into a root pitch class and a quality. In *triad mode*, qualities are reduced to the four triad types $\{M, m, dim, aug\}$; extensions are projected onto their underlying triad (e.g., $G^7 \mapsto G$, $m7b5 \mapsto dim$). In *seventh mode*, qualities map onto six four-note types $\{maj7, 7, m7, m7b5, dim7, mMaj7\}$ with semitone templates such as $maj7 = \{0, 4, 7, 11\}$.

Definition 1 (Voicing). *Given a chord with ordered tone arrangement $(p_1, \dots, p_k)$ (pitch classes, low voice first) and a register window $[\ell, h]$ in MIDI numbers, a voicing is a tuple $v = (v_1 < v_2 < \dots < v_k)$*

of MIDI notes such that $v_i \equiv p_i \pmod{12}$, $\ell \leq v_i \leq h$, and $v_{i+1} - v_i \leq 12$ for all i .

The strict-ascent and octave-spacing constraints keep voicings playable and compact; the register window defaults to $[40, 76]$ (E_2 – E_5) on desktop and $[48, 72]$ on mobile devices, whose speakers reproduce low fundamentals poorly.

The tone arrangements themselves come from small stylistic vocabularies. For triads, the $3! = 6$ permutations of $\{\text{root, third, fifth}\}$ split into three *close-position* arrangements (the root position and two inversions) and three *open* (spread) arrangements. For seventh chords, the four cyclic rotations of the tertian stack $(1, 3, 5, 7)$ give the close-position inversions, and applying the *drop-2* transform—lowering the second-from-top voice by an octave, a staple of jazz arranging [9]—to each rotation gives four open voicings, for eight arrangements total. Users select close, open, or all.

3.2 Voice-leading cost

Let $u, v \in \mathbb{Z}^k$ be voicings of consecutive chords. Chord Flow uses the per-voice weighted taxicab displacement

$$d_w(u, v) = \frac{1}{|A|} \sum_{i=1}^k w_i |u_i - v_i|, \quad w_i = \begin{cases} 1 & i \in A \\ \varepsilon & i \notin A \end{cases} \quad (1)$$

where $A \subseteq \{1, \dots, k\}$ is the user-selected set of *active* voices (bass/middle/high for triads; bass/tenor/alto/soprano for sevenths) and $\varepsilon = 10^{-3}$. Setting A to a strict subset optimizes the selected voices—e.g., the smoothest possible bass line—while the ε weight on inactive voices acts as a lexicographic tie-breaker that still prefers economical motion elsewhere. With $A = \{1, \dots, k\}$, Eq. (1) is the standard smoothness (L^1) measure of voice-leading size [4] up to the constant factor $1/k$.

3.3 Objective

Given a progression $c_1, \dots, c_n$ with voicing sets $V_1, \dots, V_n$, the task is to choose $x_i \in V_i$ minimizing total motion

$$\min_{x_1 \in V_1, \dots, x_n \in V_n} \sum_{i=1}^{n-1} d_w(x_i, x_{i+1}) + d_w(x_0, x_1), \quad (2)$$

where the extra term involving a *virtual predecessor* x_0 encodes a boundary condition discussed next. Because song forms are typically played in a loop, the transition from the final chord back to the first matters musically: an optimal open-boundary solution

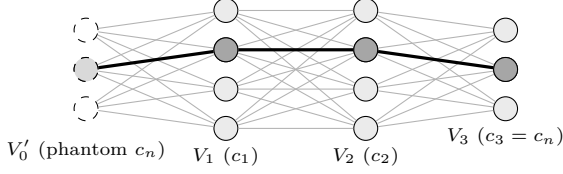


Figure 1: The layered voicing graph for a three-chord progression. Each layer contains the register-specific voicings of one chord; the dashed *phantom* layer replicates the final chord’s voicings as zero-cost sources, so the optimal path (bold) begins with an opening voicing that is smoothly reachable from some voicing of the ending chord.

may end somewhere that clashes with the top of the form. Chord Flow therefore sets x_0 to range over the voicings of the *final* chord c_n , so the chosen opening voicing is one that is reachable smoothly *from* some voicing of the ending chord. This is a relaxation of the fully cyclic objective (which would require $x_0 = x_n$); §7 discusses the gap.

4 Method

4.1 Layered voicing graph

Chord Flow instantiates objective (2) as a graph (Figure 1). For each position i , every arrangement of c_i in the active vocabulary is expanded, by depth-first search over octave placements, into all voicings satisfying Definition 1; these become the nodes of layer i . A complete bipartite set of directed edges connects layer i to layer $i+1$, weighted by Eq. (1). Finally a phantom layer containing the voicings of c_n is prepended and connected to layer 1.

Anti-stasis filter. When consecutive chords share pitch-class content (repeated chords, or enharmonically equivalent symbols), the graph contains transitions of cost ≈ 0 in which the voicing simply does not move. Musically this is usually undesirable—comping should breathe. Chord Flow suppresses any edge whose cost falls below a threshold $\tau = 10^{-4}$ *provided* the source node has at least one alternative edge with genuine motion; if a node’s only continuations are static, a single static edge with floor weight τ is retained so the graph stays connected. Moving edges additionally carry a constant offset $\delta = 10^{-3}$; since every source-to-sink path crosses the same number of layers, this uniform offset leaves the argmin unchanged and serves only as a regularizing floor.

Algorithm 1 Optimal voice-leading sequence

Require: chords $c_1..c_n$, register $[\ell, h]$, vocabulary T , active set A

- 1: $V_i \leftarrow \text{ENUMERATEVOICINGS}(c_i, T, [\ell, h])$ for all i
 - 2: $V'_0 \leftarrow \text{ENUMERATEVOICINGS}(c_n, T, [\ell, h])$ $\triangleright$ phantom
 - 3: $E \leftarrow \{(u, v, d_w(u, v)) : u \in V_i, v \in V_{i+1}\} \cup (V'_0 \times V_1)$
 - 4: $E \leftarrow \text{ANTISTASISFILTER}(E, \tau, \delta)$
 - 5: $\text{dist}[u] \leftarrow 0$ for $u \in V'_0$; ∞ otherwise
 - 6: run Dijkstra from all of V'_0 simultaneously
 - 7: $x^* \leftarrow \arg \min_{v \in V_n} \text{dist}[v]$
 - 8: **return** path to x^* , excluding the phantom node
-

4.2 Shortest-path decoding

All phantom nodes are initialized at distance zero and Dijkstra’s algorithm [5] (valid since all weights are nonnegative) is run to every node; the answer is the reconstructed path to the cheapest node in the final layer, with the phantom node dropped (Algorithm 1). Because the graph is a layered DAG, plain dynamic programming in topological order would suffice, with cost $O(\sum_i |V_i| |V_{i+1}|)$; Dijkstra was chosen for implementation simplicity and is equally exact.

Proposition 1. *Algorithm 1 returns a global minimizer of objective (2) over the filtered edge set.*

Proof sketch. Edge weights are nonnegative, so Dijkstra computes exact single-source distances; initializing all phantom nodes at distance zero implements the minimum over the virtual predecessor $x_0 \in V'_0$, and taking the cheapest final-layer node implements the outer minimization. $\square$

4.3 Complexity

Let $V = \max_i |V_i|$ and n the progression length. The graph has $O(nV)$ nodes and $O(nV^2)$ edges. In practice V is small: an arrangement vocabulary has 3–8 entries and each admits only a handful of octave placements inside a three-octave window, giving $V \approx 5$ –18 (Table 1). Even a 32-bar form with two chords per bar yields a graph of a few thousand edges, and the whole pipeline—parsing, enumeration, graph construction, decoding—completes in milliseconds in the browser, fast enough to re-solve interactively whenever the user toggles a voicing option or voice selection.

Table 1: Measured graph sizes and optima for the ii–V–I progression in C major, register [40, 76], all voices active. Node counts are per layer; phantom-layer size equals the final layer’s.

Mode	Vocabulary	Nodes/layer	Edges
Triads	close	7, 7, 8	161
Triads	open	5, 5, 6	85
Triads	all	12, 12, 14	480
Sevenths	all (close + drop-2)	16, 16, 18	832

5 Worked Example

On $Dm \rightarrow G \rightarrow C$ with close triads, the system returns

$$\underbrace{\{D_3, F_3, A_3\}}_{Dm} \rightarrow \underbrace{\{D_3, G_3, B_3\}}_{G/D} \rightarrow \underbrace{\{E_3, G_3, C_4\}}_{C/E},$$

with total motion of 4 then 3 semitones: the bass holds as a common tone into the second-inversion G , then each voice resolves by step. On the seventh-chord version $Dm^7 \rightarrow G^7 \rightarrow Cmaj^7$ it returns

$$\begin{aligned} \{D_3, F_3, A_3, C_4\} &\rightarrow \{D_3, F_3, G_3, B_3\} \\ &\rightarrow \{C_3, E_3, G_3, B_3\}, \end{aligned}$$

which is precisely the guide-tone voice leading taught in jazz harmony: the seventh of each chord resolves down by semitone into the third of the next ($C \rightarrow B$, $F \rightarrow E$) while common tones hold. That the textbook solution emerges from a generic displacement metric, with no hand-coded resolution rules, is a small empirical echo of the claim that parsimonious voice leading underlies much of tonal syntax [2].

6 System Implementation

Chord Flow is a single-page application written in TypeScript with React and built with Vite; the optimizer described above is a pure, dependency-free module of roughly 300 lines. Around it the application provides:

- **Input.** Free-text chord entry plus a built-in library of 1,695 jazz standard chord charts; progressions are also shareable via URL encoding.
- **Playback.** Tone.js [10] drives polyphonic synthesis with selectable instruments, chord or arpeggio articulation, and adjustable duration; audio is mixed to mono for consistent playback across devices.
- **Visualization.** An interactive piano keyboard highlights the sounding voicing in real time, and

abcjs [11] renders the optimized sequence as engraved notation with music-theoretically correct enharmonic spelling (the optimizer works on MIDI integers; a spelling pass restores the notated root’s accidentals).

- **Export.** One-click WAV rendering (offline synthesis) and MIDI file export.

All computation is client-side; there is no server component, which makes the tool trivially deployable (it is hosted on GitHub Pages) and usable offline once loaded.

7 Discussion and Future Work

Cyclic relaxation. The phantom layer makes the opening voicing smoothly reachable from *some* voicing of the final chord, but does not constrain the decoder to actually *end* on that voicing, so the loop seam is optimized only approximately. The exact cyclic optimum can be recovered by solving one shortest-path problem per final-layer voicing x_n (pinning $x_0 = x_n$) and taking the best, at a cost factor of $|V_n|$; given the millisecond solve times this is practical and planned.

Cost model. The taxicab metric with uniform voice weights is a deliberate simplification. It ignores classical prohibitions (parallel fifths/octaves), voice-crossing penalties between chords, register preferences, and idiomatic hand constraints. All of these are edge-local and therefore fold directly into the weight function without changing the algorithm; exposing them as user-tunable penalties—or learning them from corpora of transcribed comping—is an attractive direction.

Vocabulary. Extended and altered harmonies are currently projected onto triads or one of six seventh-chord types. Supporting rootless voicings, upper-structure triads, and 9th/11th/13th extensions enlarges the node sets but leaves the framework untouched, since the graph is agnostic to what a “voicing” contains.

Evaluation. Beyond the qualitative agreement with pedagogy shown in §5, a systematic comparison against human-arranged voice leadings (e.g., transcribed piano comping) would quantify how far a pure displacement objective goes, and where learned or rule-based corrections are needed.

8 Conclusion

Chord Flow demonstrates that globally optimal voice leading over realistic voicing vocabularies is a small, fast, and exactly solvable problem—a layered shortest path—and that wrapping the solver in an interactive, zero-install web instrument makes it useful both as a practice companion and as a pedagogical lens: users can *hear* what a displacement metric prefers, toggle voices and vocabularies, and watch canonical resolutions emerge from optimization rather than rules. The system is open source under the MIT license, and we hope it serves as a base for the richer cost models and vocabularies sketched above.

References

- [1] D. Huron. Tone and voice: A derivation of the rules of voice-leading from perceptual principles. *Music Perception*, 19(1):1–64, 2001.
- [2] D. Tymoczko. The geometry of musical chords. *Science*, 313(5783):72–74, 2006.
- [3] C. Callender, I. Quinn, and D. Tymoczko. Generalized voice-leading spaces. *Science*, 320(5874):346–348, 2008.
- [4] D. Tymoczko. *A Geometry of Music: Harmony and Counterpoint in the Extended Common Practice*. Oxford University Press, 2011.
- [5] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.
- [6] K. Ebcioglu. An expert system for harmonizing four-part chorales. *Computer Music Journal*, 12(3):43–51, 1988.
- [7] F. Pachet and P. Roy. Musical harmonization with constraints: A survey. *Constraints*, 6(1):7–19, 2001.
- [8] M. Allan and C. K. I. Williams. Harmonising chorales by probabilistic inference. In *Advances in Neural Information Processing Systems 17*, 2005.
- [9] M. Levine. *The Jazz Theory Book*. Sher Music, 1995.
- [10] Y. Mann. Tone.js: A Web Audio framework for interactive music in the browser. <https://tonejs.github.io/>.
- [11] P. Rosen and G. Dyke. abcjs: JavaScript library for rendering standard music notation. <https://www.abcjs.net/>.